IEEE 752 Z-Field Theory, Implementation and Specification Guidelines

With PMBOK for Blooming Directed Graph Tries and Anchor Root Builds

Data Structures & Blooming Directed Graph Specification

Contents

I IEEE 752 Z-Field Theory and Specification Guidelines 5

1 IEEE 752 Z-Field Theory 7
1.1 Scope and Definitions 7
1.1.1 Z-Field (Zed) 7
1.1.2 Integer (Z-Field) Rules 7
1.1.3 Floating-Point (When Used) 8
1.2 Language Mapping 8

2 IEEE 752 Implementation and Specification Guidelines 9
2.1 Specification Guidelines 9
2.1.1 Schema Compliance 9
2.1.2 Implementation Checklist 9
2.2 References 10

II PMBOK and Blooming Directed Graph Tries 11

3 Using PMBOK with IEEE 752 and Blooming Directed Graph Systems 13
3.1 PMBOK Knowledge Areas and the Z-Field Build 13
3.1.1 Integration Management 13
3.1.2 Scope and WBS 13
3.1.3 Quality and Compliance 14
3.1.4 Risk and Stakeholders 14

4 Blooming Directed Graph Tries 15
4.1 Blooming Directed Graph 15
4.1.1 Graph Structure and Z-Field 15
4.2 Tries over Blooming Directed Graphs 15
4.3 Cross-Repository and Cross-Language Consistency 16

5 Anchor Root Tries of Blooming Directed Graphs with Filter Agents and Trigger Processes 17

5.1 Root Anchor Object 17
5.2 Filter Agents (Collections) 17
5.3 Trigger Process Agent 18
5.4 Build and PMBOK 18

Part I

IEEE 752 Z-Field Theory and Specification Guidelines

Chapter 1

IEEE 752 Z-Field Theory

1.1 Scope and Definitions

The **IEEE 752 64-bit geometrica Z-field (zed)** specifies numeric representation for structural and geometric data in data structures and graph systems. The Z-field denotes the integer domain $\mathbb{Z}$; all geometric, structural, and index-like numeric values that are integers *must* use fixed-width 64-bit integer representation where the schema or API specifies a numeric type.

1.1.1 Z-Field (Zed)

- **Z-field**: The integer domain $\mathbb{Z}$. Timestamps (epoch milliseconds), counts, sizes, indices, and dimensions use 64-bit width.
- **64-bit**: All such integer fields use **64-bit** width (signed or unsigned as specified per field).
- **Geometrica**: Applies to geometric/structural uses in graph nodes, edges, tries, and trie-based anchor structures.

1.1.2 Integer (Z-Field) Rules

Context	Type	Notes
Timestamps (epoch ms)	unsigned 64-bit	e.g. `created_at_epoch_ms`
Counts, sizes, indices	64-bit (signed/unsigned)	`uint64` non-negative; `int64` when sign needed
Identity ordinals/slots	as in schema	e.g. hex $0x00$–$0x63$ (0x64 slots)

Table 1.1: IEEE 752 Z-field integer usage.

No 32-bit integer types shall be used for timestamp, count, index, or size fields in the geometrica Z-field. Serialization in Protobuf, JSON, or Avro

shall use the 64-bit type (`uint64`, `int64`, `long`) as defined in the schema.

1.1.3 Floating-Point (When Used)

Where floating-point is required (e.g. priorities, weights, metrics), use **IEEE 754-2019** (or equivalent) **binary64** (64-bit double precision). Do not use 32-bit float for geometrica or structural numeric fields when 64-bit is available.

1.2 Language Mapping

Implementations shall follow the language mapping for 64-bit unsigned (Z-field), 64-bit signed, and 64-bit float (IEEE 754) as specified in the repository `IEEE_752_64BIT_GEOMETRICA_Z_FIELD_ZED_SPECIFICATIONS` document (Swift: `UInt64`/`Int64`; C#: `ulong`/`long`; Java: `long`; Python: `int`/`float`; etc.).

Chapter 2

IEEE 752 Implementation and Specification Guidelines

2.1 Specification Guidelines

2.1.1 Schema Compliance

- **Protobuf**: Use `uint64`/`int64` for timestamp, count, index, and size fields. No `uint32`/`int32` for such geometrica fields.
- **JSON Schema**: Use `"type": "integer"` with `"minimum"`/`"maximum"` or format hint for 64-bit range where applicable.
- **Avro**: Use `long` for 64-bit integers; `double` for 64-bit float.

2.1.2 Implementation Checklist

When implementing data structures that participate in the Z-field:

1. Identify all numeric fields that represent timestamps, counts, indices, or sizes.
2. Map each to `UInt64`, `Int64`, or `Double` (IEEE 754) per the language mapping.
3. Ensure wire formats (e.g. Gated Loopback Flower `created_at_epoch_ms`) use `uint64`.
4. Document any range limits where the runtime does not provide native 64-bit types.

2.2 References

- Gated Loopback Flower: `created_at_epoch_ms` is `uint64` (compliant).
- Node identity hex space: $0x00$–$0x63$ (0x64 slots) is separate from the 64-bit Z-field; both apply.
- IEEE 754: ISO/IEC/IEEE 60559 (binary64) for double-precision floating-point when used.

Part II

PMBOK and Blooming Directed Graph Tries

Chapter 3

Using PMBOK with IEEE 752 and Blooming Directed Graph Systems

3.1 PMBOK Knowledge Areas and the Z-Field Build

The **PMBOK** (Project Management Body of Knowledge) provides a framework for managing projects. When applying PMBOK to the implementation and specification of systems that use the IEEE 752 Z-field and Blooming Directed Graph Tries (including anchor root Tries with filter agents and trigger processes), the following alignments apply.

3.1.1 Integration Management

- **Project charter and plan**: Define scope to include IEEE 752 Z-field compliance for all numeric fields in data structures, and the deployment of the root anchor `Trie_of_blooming_directed_graphs_with_agents_and_filters` across repositories and languages.
- **Unified structure**: The anchor root Trie is the single entry point; integration of filter agents and trigger process agents under this root aligns with PMBOK integration management (unified direction and change control).

3.1.2 Scope and WBS

- **Scope**: (1) IEEE 752 Z-field implementation in all relevant data structures; (2) Blooming directed graph as nodes (repos) and edges (e.g. to `github_actions`); (3) Trie-based root anchor; (4) Filter agents (collections) and trigger process agents per platform.

- **Work breakdown**: Decompose by repository, by language (Swift, Objective-C, Ruby, TypeScript, Python, C#, Java, Bash, Zsh, Git, GitHub, HPCC), and by component (root anchor, filter agents, trigger process agent, Z-field types).

3.1.3 Quality and Compliance

- **Quality standards**: Conformance to IEEE 752 specification (64-bit for timestamps, counts, indices); consistent semantics for filter and trigger actions across 12 languages and 4 environments.
- **Verification**: Tests (e.g. DataStructuresIEEE752 Swift package, unit tests for Trie root and agents) verify Z-field types and anchor/filter/trigger behavior.

3.1.4 Risk and Stakeholders

- **Risk**: Drift from 64-bit Z-field (e.g. use of 32-bit); inconsistent naming or behavior of anchor Trie, filter agents, or trigger processes across repos.
- **Stakeholders**: Consumers of the 14 repositories and 12 language implementations; automation and tooling that assume the root anchor and IEEE 752 types.

This chapter provides the bridge between PMBOK practices and the technical implementation of the IEEE 752 Z-field and Blooming Directed Graph Trie build described in the following chapters.

Chapter 4

Blooming Directed Graph Tries

4.1 Blooming Directed Graph

A **blooming-directed-graph** is a directed graph whose nodes are repositories (e.g. under `~/projects`) and whose edges are directed (e.g. repo → `github_actions`). The term is specific and distinct from a generic "blooming graph." Each repository is a node; edges link nodes to automation targets such as GitHub Actions.

4.1.1 Graph Structure and Z-Field

Graph edges that carry temporal or structural metadata shall use IEEE 752 Z-field types. For example:

- `created_at_epoch_ms`: `uint64` (epoch milliseconds).
- Counts of nodes or edges: `UInt64` or equivalent in the implementation language.

4.2 Tries over Blooming Directed Graphs

A **Trie** (prefix trie) can index blooming directed graphs by key (e.g. repo name or context-key prefix). Each key maps to a value that references:

1. A **blooming-directed-graph** node (or subgraph),
2. Associated **filter agents** (collections that filter nodes by receiver, action_where, action_client, repo name),
3. Associated **trigger process agent** (which triggers workflow_dispatch, git actions, fetch_merge, push_merge, pull_merge, job_submit, run_script, notify).

Such a Trie provides a single, consistent index for resolving a key to the corresponding graph context and agents. Implementation of the Trie (insert, get, starts_with) and of the stored values shall use 64-bit types for any counts or timestamps per IEEE 752.

4.3 Cross-Repository and Cross-Language Consistency

Blooming directed graph Tries are implemented in all 12 languages (Swift, Objective-C, Ruby, TypeScript, Python, C#, Java, Bash, Zsh, Git, GitHub, HPCC) and are present in all 14 repositories. This ensures that tooling and automation can assume a single conceptual structure: a Trie of blooming directed graphs with agents and filters, with Z-field-compliant numeric fields throughout.

Chapter 5

Anchor Root Tries of Blooming Directed Graphs with Filter Agents and Trigger Processes

5.1 Root Anchor Object

The **root anchor object** in every repository and language is:

`Trie_of_blooming_directed_graphs_with_agents_and_filters`

It is the single entry point: a Trie whose keys are repo names or context-key prefixes and whose values hold references to:

- the **blooming-directed-graph** node (or graph reference),
- the **filter agents** (collection) for that context,
- the **trigger process agent** for that context.

Everything else (blooming-directed-graph, filter agents, trigger process agent) hangs off this root. The anchor root Trie is implemented in all 12 languages and deployed in all 14 repositories.

5.2 Filter Agents (Collections)

Blooming directed graph filter agents are collections that take the graph (nodes and edges) and optional parameters and return a filtered subset. Common semantics across platforms:

- `filter_by_receiver(nodes, ‘‘github’’ | ‘‘hpcc’’)`,
- `filter_by_action_where(nodes, ‘‘github’’ | ‘‘hpcc’’)`,

- `filter_by_action_client(nodes, ‘‘macbook’’ | ‘‘rockydesktop’’)`,
- `filter_by_repo_name(nodes, pattern)`,
- `filter_by_context_key(nodes, key)`.

Implementations live under `blooming_directed_graph_agents/filter_agents/<platform>/`. Any numeric fields used in filtering (e.g. counts, indices) shall conform to IEEE 752 Z-field (64-bit).

5.3 Trigger Process Agent

The **blooming directed graph trigger process agent** takes a node (or filtered set), an action type, and optional payload, and triggers a process. Action types include:

- `workflow_dispatch`, `job_submit`,
- `git_push`, `git_fetch`,
- `fetch_merge`, `push_merge`, `pull_merge`,
- `run_script`, `notify`.

Implementations live under `blooming_directed_graph_agents/trigger_process_agent/<plat` Payloads and any timestamps or counts shall use 64-bit types per IEEE 752 where applicable.

5.4 Build and PMBOK

The **build** of the anchor root Tries of Blooming Directed Graphs with filter agents and trigger processes is governed by:

1. **IEEE 752 Z-field**: All timestamps, counts, and indices in data structures and agent payloads use 64-bit types as specified.
2. **PMBOK**: Scope (repos, languages, components), WBS (by repo/language/component), quality (Z-field compliance and test coverage), and integration (single root anchor, consistent filter/trigger semantics) are managed per PMBOK practices.
3. **Deployment**: The same directory layout (`Trie_of_blooming_directed_graphs_with_ag` `blooming_directed_graph_agents/`) and the same specification documents (e.g. `IEEE_752_64BIT_GEOMETRICA_Z_FIELD_ZED_SPECIFICATIONS`, `BLOOMING_DIRECTED_GRAPH_AGENTS`, `TRIE_ROOT_ANCHOR`) are used across all 14 repositories to ensure a consistent, testable, and maintainable build.